

SOFTWARE ENGINEERING ASSIGNMENT

1. Decision Tree

Definition

A Decision Tree is a graphical representation used in software engineering to model decision-making processes and conditional logic. It illustrates decisions and their possible consequences in a tree-like structure consisting of nodes and branches.

Decision trees are commonly used during requirements analysis, system modeling, algorithm design, and business rule modeling.

Components of a Decision Tree

Root Node – Represents the initial decision point.

Decision Nodes – Points where a condition is evaluated.

Branches – Represent outcomes of a decision (e.g., Yes/No, True/False).

Leaf Nodes (Terminal Nodes) – Represent final outcomes or actions.

Structure

The structure follows a hierarchical format:

Copy code

[Condition A?]

/ \

Yes

No

/ \

[Condition B?]

Action 3

/ \

Yes

No

Action 1 Action 2

Applications in Software Engineering

Modeling complex business logic

Designing conditional program structures

Risk analysis and decision-making

Test case generation

AI and machine learning classification

Advantages

Easy to understand and visualize

Simplifies complex conditional logic

Useful for identifying missing rules

Helps in test coverage analysis

Disadvantages

Can become very large and complex

Difficult to maintain for systems with many conditions

Not suitable for highly mathematical specifications

2. Decision Table

Definition

A Decision Table is a tabular method for representing complex logical relationships between conditions and actions. It systematically lists all possible combinations of conditions and their corresponding actions.

Decision tables are widely used in requirements engineering and system design to ensure completeness and

consistency of business rules.

Structure of a Decision Table

A decision table consists of four main parts:

Condition Stub – Lists all conditions

Action Stub – Lists all possible actions

Condition Entries – Possible values of conditions (Yes/No, True/False)

Action Entries – Actions to be executed

Example Structure

Conditions

Rule 1

Rule 2

Rule 3

Rule 4

Condition A

Yes

Yes

No

No

Condition B

Yes

No

Yes

No

Actions

Action 1

X

Action 2

X

X

Action 3

X

Types of Decision Tables

Limited-entry Decision Table – Uses binary values (Yes/No).

Extended-entry Decision Table – Allows multiple values (e.g., High/Medium/Low).

Mixed-entry Decision Table – Combination of both types.

Applications in Software Engineering

Modeling business rules

Requirement validation

Complex logic implementation

Test case derivation

Ensuring completeness of system specifications

Advantages

Ensures completeness of all condition combinations

Reduces ambiguity in requirement

documentation

Easy to convert into program code

Helps detect inconsistent rules

Disadvantages

Can become large with many conditions

Difficult to manage manually for complex systems

3. Formal System Specification

Definition

Formal System Specification is the process of describing a software system using mathematical notations and formal logic to eliminate ambiguity and ensure precision.

Unlike informal specifications written in natural language, formal specifications use mathematical models, logic, and set theory to describe system behavior, data structures, and constraints.

Purpose of Formal Specification

Remove ambiguity in requirements

Provide precise system behavior
description

Enable formal verification and validation

Improve software reliability and safety

Support correctness proofs

Characteristics

Mathematically precise

Unambiguous

Consistent

Verifiable

Complete

Types of Formal Specification Approaches

Model-Based Specification

Describes system state and operations
mathematically

Examples include:

Z Notation

Vienna Development Method (VDM)

Algebraic Specification

Specifies system behavior using algebraic
equations

Focuses on abstract data types

Process-Oriented Specification

Describes interactions and communication between system components

Examples include CSP (Communicating Sequential Processes)

Example (Simple Formal Specification Concept)

Let:

$\text{Balance} \geq 0$

$\text{Withdrawal amount} \leq \text{Balance}$

Operation:

Copy code

Withdraw(amount)

Pre-condition: $\text{amount} \leq \text{Balance}$

Post-condition: $\text{Balance} = \text{Balance} - \text{amount}$

This ensures system correctness through logical constraints.

Advantages

Eliminates ambiguity in system requirements

Enables mathematical verification

Detects inconsistencies early

Essential for safety-critical systems (e.g., aviation, banking, medical software)

Disadvantages

Requires strong mathematical knowledge

Time-consuming to develop

Not always practical for small projects

Limited industry adoption compared to informal methods

Conclusion

Decision Trees provide a graphical method for modeling conditional decisions.

Decision Tables provide a structured tabular approach for representing complex business logic.

Formal System Specification ensures mathematical precision and correctness in

software requirements.

Together, these techniques improve requirement clarity, system correctness, and software quality in software engineering practice.